
```

function [f,J]=CircuitoResiduo(NodosCoords, ResistNodos, ResistCoefs,
FuentesNodos, ...
    FuentesCoefs, x, CalcJ)

%N'umero de nodos, resistencias y fuentes:
nNodos      = size(NodosCoords,1);
nResist     = size(ResistNodos,1);
nFuentes    = size(FuentesNodos,1);

%Reservamos memoria:
f           = zeros(nNodos+nFuentes+1, 1);
if CalcJ
    filas    = [];
    cols     = [];
    vals     = [];
end

%Recorremos resistencias:
for ii=1:nResist

    %Nodos de la resistencia:
    NodoA    = ResistNodos(ii,1);
    NodoB    = ResistNodos(ii,2);

    %Coeficientes de la resistencia:
    alpha    = ResistCoefs(ii,1);
    beta     = ResistCoefs(ii,2);

    %Intensidad:
    VA       = x(NodoA);
    VB       = x(NodoB);
    I        = alpha*(VA-VB) + beta*(VA-VB)^3;

    %Contribuciones de la intensidad en NodoA y NodoB:
    f(NodoA) = f(NodoA) + I;
    f(NodoB) = f(NodoB) - I;

    %Calculamos jacobiano:
    if CalcJ
        dI_dV = alpha+3*beta*(VA-VB)^2;
        filas = cat(1, filas, NodoA, NodoA, NodoB, NodoB);
        cols  = cat(1, cols,  NodoA,  NodoB,  NodoA,  NodoB);
        vals  = cat(1, vals, dI_dV, -dI_dV, -dI_dV, dI_dV);
    end
end

%Contribuciones de las fuentes de tensión:
for ii=1:nFuentes

    %Nodos de la fuente:
    NodoA      = FuentesNodos(ii,1);

```

```

    NodoB          = FuentesNodos(ii,2);

    %Intensidad que circula por la fuente:
    I              = x(nNodos+ii);

    %Contribuci'on de la fuente a las intensidades en NodoA y NodoB:
    f(NodoA)       = f(NodoA) + I;
    f(NodoB)       = f(NodoB) - I;

    %Imponemos diferencia de potencial entre n1 y n2:
    f(nNodos+ii)   = x(NodoB)-x(NodoA)-FuentesCoefs(ii);

    %Calculamos jacobiano:
    if CalcJ
        filas      = cat(1, filas, NodoA, NodoB, nNodos+ii, nNodos+ii);
        cols       = cat(1, cols, nNodos+ii, nNodos+ii, NodoB, NodoA);
        vals       = cat(1, vals, 1.0, -1.0, 1.0, -1.0);
    end

end

%Contribuci'on de la corriente de fuga a tierra:
f(1)              = f(1) + x(nNodos+nFuentes+1);
f(nNodos+nFuentes+1) = x(1);
if CalcJ
    filas          = cat(1, filas, 1, nNodos+nFuentes+1);
    cols           = cat(1, cols, nNodos+nFuentes+1, 1);
    vals           = cat(1, vals, 1.0, 1.0);
end

%Construimos jacobiano:
if CalcJ
    J              = sparse(filas, cols, vals, nNodos+nFuentes+1,
nNodos+nFuentes+1);
else
    J              = NaN;
end

end

```

Published with MATLAB® R2025a

```
function FuentesNodos=CrearFuentes1(M,N)

%     FuentesNodos    = zeros(2*(N-1),2);
%
%     %Fuentes inferiores:
%     kk              = 0;
%     for ii=1:M-1
%         kk          = kk+1;
%         FuentesNodos(ii,1) = ii*N+1;
%         FuentesNodos(ii,2) = (ii-1)*N+1;
%     end
%
%     %Fuentes superiores:
%     for ii=M:-1:2
%         kk          = kk+1;
%         FuentesNodos(kk,1) = (ii-2)*N+N;
%         FuentesNodos(kk,2) = (ii-1)*N+N;
%     end

%     FuentesNodos      = zeros(2, 2);
%     FuentesNodos(1,1) = N;
%     FuentesNodos(1,2) = 1;
%     FuentesNodos(2,1) = (M-1)*N+1;
%     FuentesNodos(2,2) = M*N;

FuentesNodos      = zeros(2, 2);
FuentesNodos(1,1) = 1;
FuentesNodos(1,2) = (M-1)*N+1;
FuentesNodos(2,1) = M*N;
FuentesNodos(2,2) = N;
```

```
end
```

Published with MATLAB® R2025a

```
function FuentesCoefs = CrearFuentes2(M,N)

%     nF           = 2*(M-1);
%     FuentesCoefs = 1.5 + 0.5*cos(1:nF)';
%     warning('Review')

    nF           = 2;
    FuentesCoefs = 2.5 + 0.5*cos(1:nF).';

end
```

Published with MATLAB® R2025a

```
function NodosCoords = CrearNodos(M,N)

%Dimensiones:
Lx          = 2.0;
Ly          = 3.0;

%Mallado:
xv          = linspace(0.0, Lx, M);
yv          = linspace(0.0, Ly, N);

%Vector desplazamiento:
lambdax     = 3*pi;
lambday     = 3*pi;
Ax          = 0.0*0.5/( lambdax/Lx );
Ay          = 0.0*0.5/( lambday/Ly );

%Definimos nodos:
NodosCoords = zeros(M*N,2);
for ii=1:M
    for jj=1:N
        kk      = (ii-1)*N+jj;
        x_i     = xv(ii);
        y_j     = yv(jj);
        u_ij    = Ax * cos(lambdax*x_i/Lx)*...
                    (y_j)*(Ly-y_j)/(0.25*Ly^2);
        v_ij    = Ay * (x_i)*(Lx-x_i)/(0.25*Lx^2)*...
                    cos(lambday*y_j/Ly);
        NodosCoords(kk,1) = x_i + u_ij;
        NodosCoords(kk,2) = y_j + v_ij;
    end
end
end

end
```

Published with MATLAB® R2025a

```
function ResistNodos=CrearResistencias1(M,N)

%Resistencias verticales:
ResistNodos = zeros(M*(N-1)+(M-1)*(N-1),2);
for ii=1:M
    for jj=1:N-1
        kResist = (ii-1)*(N-1)+jj;
        ResistNodos(kResist,1) = (ii-1)*N+jj;
        ResistNodos(kResist,2) = (ii-1)*N+jj+1;
    end
end

%Resistencias diagonales:
for ii=1:M-1
    for jj=1:N-1
        kResist = M*(N-1)+(ii-1)*(N-1)+jj;
        ResistNodos(kResist,1) = (ii-1)*N+jj+1;
        ResistNodos(kResist,2) = ii*N+jj;
    end
end

end
```

Published with MATLAB® R2025a

```
function ResistCoefs = CrearResistencias2(M,N)
```

```
    %alpha_k = 1+0.5*cos(k)
```

```
    %beta_k = 1+0.5*sin(k)
```

```
    nR      = M*(N-1) + (M-1)*(N-1);
```

```
    ResistCoefs = cat(2, 1+0.5*cos(1:nR).', 1+0.5*sin(1:nR).');
```

```
end
```

Published with MATLAB® R2025a

```
function Wv=PotenciasFuentes(nNodos, FuentesNodos, FuentesCoefs, x)

%Reservamos memoria:
nResist      = size(FuentesNodos,1);
Wv           = zeros(nResist,1);
for ii=1:nResist

    %Nodos de la fuente:
    NodoA     = FuentesNodos(ii,1);
    NodoB     = FuentesNodos(ii,2);

    %Intensidad:
    I         = x(nNodos+ii);

    %Potencia:
    Wv(ii)    = FuentesCoefs(ii)*I;

end

end
```

Published with MATLAB® R2025a

```
function Wv=PotenciasResist(ResistNodos, ResistCoefs, x)

%Reservamos memoria:
nResist      = size(ResistNodos,1);
Wv           = zeros(nResist,1);
for ii=1:nResist

    %Nodos de la resistencia:
    NodoA     = ResistNodos(ii,1);
    NodoB     = ResistNodos(ii,2);

    %Coeficientes de la resistencia:
    alpha     = ResistCoefs(ii,1);
    beta      = ResistCoefs(ii,2);

    %Intensidad:
    VA        = x(NodoA);
    VB        = x(NodoB);
    I         = alpha*(VA-VB) + beta*(VA-VB)^3;

    %Potencia:
    Wv(ii)    = (VA-VB)*I;

end

end
```

Published with MATLAB® R2025a

```

function Practica1(C1,C2)
%C1, C2: par'ámetros que determinan el tama'no del circuito.

%Geometr'ia del circuito:
NodosCoords    = CrearNodos(C1,C2);
ResistNodos     = CrearResistencias1(C1,C2);
FuentesNodos    = CrearFuentes1(C1,C2);
%Representaci'on:
RepresentarCircuito(NodosCoords, ResistNodos, FuentesNodos)

%Coeficientes que caracterizan las resistencias y las fuentes:
ResistCoefs     = CrearResistencias2(C1,C2);
FuentesCoefs    = CrearFuentes2(C1,C2);

%Condic'i'on inicial:
nN              = size(NodosCoords,1);
nR              = size(ResistNodos,1);
nF              = size(FuentesNodos,1);
x0              = zeros(nN+nF+1,1);

%Resolvemos con Anderson:
tic
[~,J0]          = CircuitoResiduo(NodosCoords, ResistNodos, ResistCoefs,
...
                                FuentesNodos, FuentesCoefs, x0, true);
[L,U,P,Q]       = lu(J0);
function g=gfun(x)
    [f,~]        = CircuitoResiduo(NodosCoords, ResistNodos,
ResistCoefs, ...
                                FuentesNodos, FuentesCoefs, x, false);
    g            = Q*(U\(L\(P*f)));
end
[x,nIters,flag] = Anderson1(@gfun, x0, 1e-8, 100, 10);
toc
if flag<0
    warning('El metodo no ha convergido')
end
display(nIters)

%Corriente de fuga:
I_fuga         = x(end);
display(I_fuga)

%Potencias:
Wv             = PotenciasResist(ResistNodos, ResistCoefs, x);
Wtotal         = sum(Wv);
display(Wtotal)
Wv             = PotenciasFuentes(nN, FuentesNodos, FuentesCoefs, x);
Wtotal         = sum(Wv);
display(Wtotal)

%Representar potencial:

```

```
RepresentarPotencial(NodosCoords, x)
end
```

Published with MATLAB® R2025a

```
function RepresentarCircuito(NodosCoords, ResistNodos, FuentesNodos)

%Abrimos figura:
figure()

%Representar resistencias:
nR = size(ResistNodos,1);
for ii=1:nR
    NodoA = ResistNodos(ii,1);
    NodoB = ResistNodos(ii,2);
    plot([NodosCoords(NodoA,1),NodosCoords(NodoB,1)], ...
         [NodosCoords(NodoA,2),NodosCoords(NodoB,2)], 'o-b')
    hold on
end

%Representar fuentes:
nF = size(FuentesNodos,1);
for ii=1:nF
    NodoA = FuentesNodos(ii,1);
    NodoB = FuentesNodos(ii,2);
    plot([NodosCoords(NodoA,1),NodosCoords(NodoB,1)], ...
         [NodosCoords(NodoA,2),NodosCoords(NodoB,2)], 's-r')
    hold on
end

%Se'nalar nodo 1 (conectado a tierra):
plot(NodosCoords(1,1), NodosCoords(1,2), 'x-k')

end
```

Published with MATLAB® R2025a

```
function RepresentarPotencial(NodosCoords, x)

    %N'umero de nodos:
    nNodos      = size(NodosCoords,1);

    %Vector con las tensiones:
    Vv          = x(1:nNodos);

    figure()
    plot3(NodosCoords(:,1), NodosCoords(:,2), Vv, '.b')
    xlabel('x')
    ylabel('y')
    zlabel('V')

end
```

Published with MATLAB® R2025a